

AMAT371 "Computing for Mathematicians"
(Matlab Sessions, Winter Block Week, 2006
Instructor: A. Swishchuk)

**1 Lecture 4: Ordinary Differential Equations (ODE)
and Boundary Value Problems (BVP) with Mat-
lab and Applications.**

1.1 Outline

- ODE with Matlab
- Application of ODE: Pursuit Problem with Event Location
- BVP with Matlab
- Application of BVP: Water droplet on a flat surface

1.2 ODE with Matlab

The general form of a call to ODE is

$$[t, y] = \text{ode45}(@\text{fun}, tspan, yzero, options, p1, p2, \dots).$$

$p1, p2, \dots$ represent problem parameters that are passed on to the function **fun**.

Matlab's ODE solvers may be found in Lecture 6, Fall Block Week, 2004, AMAT371, M1, web page:

<https://www.math.ucalgary.ca/~aswish/lec6matlab.pdf>

1.3 Application of ODE: Pursuit Problem with Event Location

Consider a pursuit problem. Suppose that a rabbit follows a predefined path $(r_1(t), r_2(t))$ in the plane, and that a fox chases the rabbit in such a way that

(a) at each moment the tangent of the fox's path points towards the rabbit and

(b) the speed of the fox is some constant k times the speed of the rabbit.

Then the path $(y_1(t), y_2(t))$ of the fox is determined by the ODE

$$\begin{aligned}\frac{d}{dt}y_1(t) &= s(t)(r_1(t) - y_1(t)), \\ \frac{d}{dt}y_2(t) &= s(t)(r_2(t) - y_2(t)),\end{aligned}$$

where

$$s(t) = \frac{k\sqrt{(\frac{d}{dt}r_1(t))^2 + (\frac{d}{dt}r_2(t))^2}}{\sqrt{(r_1(t) - y_1(t))^2 + (r_2(t) - y_2(t))^2}}.$$

The system of ODE above becomes ill-defined if the fox approaches the rabbit. We let the rabbit follows an outward spiral,

$$\begin{bmatrix} r_1(t) \\ r_2(t) \end{bmatrix} = \sqrt{1+t} \begin{bmatrix} \cos(t) \\ \sin(t) \end{bmatrix},$$

and start the fox at $y_1(0) = 3$, $y_2(0) = 0$. The function **fox1** implements the ODE, with k set to 0.75.

Create the following M-file **fox1.m**:

```
function yprime = fox1(t,y)
%FOX1 Fox - rabbit pursuit simulation
% YPRIME = FOX1(T,Y).
k = 0.75;
r = sqrt(1+t) * [cos(t); sin(t)];
r_p = (0.5/sqrt(1+t)) * [cos(t) - 2 * (1+t) * sin(t); sin(t) + 2 * (1+t) * cos(t)];
dist = norm(r - y);
if dist > 1e-4
    factor = k * norm(r_p)/dist;
    yprime = factor * (r - y);
else
    error('ODE model ill-defined.')
end
```

The following script calls **fox1** to produce figure of pursuit example for the phase paths of the rabbit and fox. Initial conditions are denoted by circles and the dashed and solid lines show the phase paths of the rabbit and fox, respectively.

```
>> tspan = [0 10]; yzero = [3; 0];
>> [tfox, yfox] = ode45(@fox1, tspan, yzero);
>> plot(yfox(:,1), yfox(:,2)), hold on
>> plot(sqrt(1+tfox) * cos(tfox), sqrt(1+tfox) * sin(tfox), '--')
>> plot([3 1], [0 0], 'o');
>> axis equal, axis([-3.5 3.5 -2.5 3.1])
>> legend('Fox', 'Rabbit', 0), hold off
```

The implementation above is unsatisfactory for $k > 1$, that is, when the fox is faster than the rabbit. In this case, if the rabbit is caught within the specified time interval then no solution is displayed. It would be more natural to ask **ode45** to return with the computed solution if the fox and rabbit become close. The following script files **fox2.m**

```
function yprime = fox2(t,y,k)
%FOX1 Fox - rabbit pursuit simulation
%      with relative speed parameter
%      YPRIME = FOX2(T,Y,K).
r = sqrt(1+t) * [cos(t); sin(t)];
r_p = (0.5/sqrt(1+t)) * [cos(t) - 2*(1+t)*sin(t); sin(t) + 2*(1+t)*cos(t)];
dist = max(norm(r-y), 1e-6);
factor = k * norm(r_p)/dist;
yprime = factor * (r - y);
```

and **events.m**

```
function [value, isterminal, direction] = events(t,y,k)
%EVENTS Events function for FOX2.
%      Locate when fox is close to rabbit
r = sqrt(1+t) * [cos(t); sin(t)];
value = norm(r-y) - 1e-4; %Fox close to rabbit.
isterminal = 1; %Stop integration.
direction = -1; %Value must be decreasing through zero.
```

uses the functions **fox2** and **events** to produce figure about this event. The initial conditions and the rabbit's path are the same as in previous example.

```
>> k = 1.1;
>> tspan = [0; 10]; yzero = [3; 0];
>> options = odeset('RelTol', 1e-6, 'AbsTol', 1e-6, 'Events', @events);
>> [tfox, yfox, te, ye, ie] = ode45(@fox2, tspan, yzero, options, k);
>> plot(yfox(:,1), yfox(:,2)), hold on
>> plot(sqrt(1+tfox) * cos(tfox), sqrt(1+tfox) * sin(tfox), '--')
>> plot([3 1], [0 0], 'o'), plot(yfox(end,1), yfox(end,2), '*');
>> axis equal, axis([-3.5 3.5 -2.5 3.1])
>> legend('Fox', 'Rabbit', 0), hold off
```

The output arguments from **ode45** are **[tfox,yfox,te,ye,ie]**. Here, **tfox** and **yfox** are the usual solution approximations, so **yfox(i)** approximates

$y(t)$ at time $t = tfox(i)$. The arguments **te** and **ye** record those t and y values at which the event(s) were recorded and, for vector valued events, **ie** specifies which component of the event occurred each time. In our example, we have

```
>> te, ye
    te =
    5.0710
    ye =
    0.8646   -2.3073
```

showing that the rabbit was captured after 5.07 time units at the point $(0.86, -2.31)$.

1.4 BVP with Matlab

In general, **bvp4c** can be called in the form

$$sol = bvp4c(@odefun, @bcfun, solinit, options, p1, p2, \dots);$$

Here, **odefun** evaluates the differential equations and **bcfun** gives the residual for the boundary conditions. The function **odefun** has the general form

$$yprime = odefun(x, y, p1, p2, \dots)$$

and **bcfun** has the general form

$$res = bcfun(ya, yb, p1, p2, \dots).$$

1.5 Application of BVP: Water droplet on a flat surface

The function **bvp4c** uses a collocation method to solve systems of ODEs in two-point boundary value form. These systems may be written

$$\frac{d}{dc}y(x) = f(x, y(x)), \quad g(y(a), y(b)) = 0.$$

Here, $y(x)$ is an unknown m -vector and f is a given function of x and y that also produces an m -vector. The given function g specifies the *boundary conditions*. Note that the independent variable was labeled t in the previous paragraph and is now labeled x . This reflects the fact that two-point boundary value problem usually arise over an interval of space rather than time.

Consider a scalar BVP describing the cross-sectional shape of a water droplet on a flat surface:

$$\frac{d^2}{dx^2}h(x) + (1 - h(x))(1 + (\frac{d}{dx}h(x))^2)^{3/2} = 0, \quad h(-1) = 0, \quad h(1) = 0.$$

Here, $h(x)$ measures the height of the droplet at point x . We set $y_1(t) = h(x)$ and $y_2(t) = dh(x)/dx$ and rewrite the equation as a system of two-first-order equations:

$$\begin{aligned} \frac{d}{dx}y_1(x) &= y_2(x), \\ \frac{d}{dx}y_2(x) &= (y_1(x) - 1)(1 + y_2(x)^2)^{3/2}. \end{aligned}$$

This system is represented by the function **yprime=drop(x,y)** below. Create the following M-file **drop.m**

```
function yprime = drop(x,y)
%DROP ODE/BVP water droplet example.
% YPRIME = DROP(X,Y) evaluates derivative.
yprime = [y(2); (y(1) - 1) * ((1 + y(2)^2)^(3/2))];
```

Boundary conditions can be encoded in the following function **dropbc.m**

```
function res = dropbc(ya,yb)
%DROPBC ODE/BVP water droplet boundary conditioins.
% RES = DROPBC(XA,YB) evaluates residual.
res = [ya(1);yb(1)];
```

As an initial guess for the solution, we use $y_1(x) = \sqrt{1 - x^2}$ and $y_2(x) = -x/(0.1 + \sqrt{1 - x^2})$. This information is set up by the function **dropinit**:

```
function yinit = dropinit(x)
%DROPINIT ODE/BVP water droplet initial guess.
% YINIT = DROPINIT(X) evaluates initial guess at X.
yinit = [sqrt(1 - x.^2); -x./(0.1 + sqrt(1 + x.^2))];
```

The following code solves the BVP and produces figure for this problem:

```
>> solinit = bvpinit(linspace(-1,1,20),@dropinit);
>> sol = bvp4c(@drop,@dropbc,solinit);
>> fill(sol.x,sol.y(1,:),[0.7 0.7 0.7])
>> axis([-1 1 0 1])
>> xlabel('x','FontSize',16)
>> ylabel('h','Rotation',0,'FontSize',16)
```